

Contents

Gemini CLI: Comprehensive Architecture & Feature Reference	2
Table of Contents	3
1. Overview	3
What Is Gemini CLI?	3
Key Facts	3
Installation	3
Authentication Options	4
2. Architecture	4
High-Level Architecture	4
Monorepo Package Structure	5
Key Architectural Decisions	5
3. Core Features	5
Code Understanding & Generation	5
File Operations	5
Shell Execution	5
Web & Search	6
Planning	6
Interaction Tools	6
Input Shortcuts	6
4. Models	6
Supported Models	6
Model Routing & Aliases	7
Thinking Configuration	7
Selecting a Model	8
Pricing & Quotas	8
5. Tool System	8
Architecture	8
Complete Built-in Tools Reference	8
Tool Confirmation Behavior	9
Tool Discovery & Management	9
Manually-Triggered Tool Shortcuts	9
6. MCP Support	9
Integration Architecture	9
Transport Mechanisms	10
Configuration	10
MCP Server Properties	10
Global MCP Settings	11
MCP Resources	11
MCP Management Commands	11
MCP Invocation in Prompts	11
7. Skills & Extensions	11
Agent Skills	11
Extensions	13
Custom Commands	13
Management	15
IDE Companion Spec	15
10. Session Management	16
Automatic Session Saving	16
Resuming Sessions	16
Session Browser Features	16
Manual Checkpoints	16
Session Retention	16
Session Limits	17
Checkpointing (File State Snapshots)	17
Parallel Sessions with Git Worktrees	17

11. Configuration	17
Configuration Hierarchy (ascending precedence)	17
Settings File Locations	17
Key Settings Categories	17
Environment Variables	19
The <code>.gemini</code> Directory	19
12. Permissions & Security	20
Approval Modes	20
Sandboxing	20
Trusted Folders	20
Policy Engine (since v0.18.0)	21
Hooks (Lifecycle Events)	21
13. Context Management	21
Context File Hierarchy (GEMINI.md)	21
Memory Management	22
Context Compression	22
Token Caching	22
Tool Output Summarization	22
<code>.geminiignore</code>	22
14. Unique Features	22
1. Google Search Grounding	23
2. Free Tier with No API Key	23
3. Plan Mode with Model Routing	23
4. Extension Ecosystem with Gallery	23
5. Interactive Shell	23
6. Shell Mode Toggle	23
7. Conversation Rewind (<code>/rewind</code>)	23
8. Multi-Directory Workspace	23
9. GitHub Action Integration	23
10. Vim Mode	23
11. A2A (Agent-to-Agent) Protocol	23
12. Custom Themes & Auto-Switching	23
13. Checkpointing with Shadow Git	24
14. Token Caching	24
15. Google Colab Integration	24
16. Multimodal Input	24
17. <code>gemini-wrapped</code>	24
15. Limitations vs Claude Code	24
What Gemini CLI Lacks Compared to Claude Code	24
Where Gemini CLI Excels Over Claude Code	24
16. Version History	25
Timeline of Key Releases	25
Key Milestones	26
Release Cadence	26
Appendix A: Quick Command Reference	26
Slash Commands	26
CLI Flags	27
Exit Codes (Headless Mode)	27
Appendix B: Comparison Matrix – Gemini CLI vs Claude Code	28

Gemini CLI: Comprehensive Architecture & Feature Reference

Last updated: 2026-03-25 **Current version:** v0.35.0 stable / v0.36.0-preview.0 **Repository:** [google-gemini/gemini-cli](#) **Stars:** ~99,000 | **Forks:** ~12,600 | **License:** Apache 2.0 **Website:** [geminicli.com](#)

Table of Contents

- 1. Overview
- 2. Architecture
- 3. Core Features
- 4. Models
- 5. Tool System
- 6. MCP Support
- 7. Skills & Extensions
- 8. Agent Capabilities
- 9. IDE Integration
- 10. Session Management
- 11. Configuration
- 12. Permissions & Security
- 13. Context Management
- 14. Unique Features
- 15. Limitations vs Claude Code
- 16. Version History

1. Overview

What Is Gemini CLI?

Gemini CLI is Google’s open-source AI coding agent that operates directly in the terminal. It provides developers with conversational access to Gemini models for code generation, editing, debugging, automation, and codebase exploration. It was created by Google as part of the `google-gemini` organization on GitHub.

Key Facts

Property	Value
First public release	June 25, 2025 (repo created April 17, 2025)
Current stable	v0.35.0 (March 2026)
Language	TypeScript (monorepo)
Runtime	Node.js (npm package: <code>@google/gemini-cli</code>)
License	Apache 2.0
Free tier	60 requests/min, 1,000 requests/day (Google account)
Context window	1M tokens (Gemini 3 models)
Release cadence	Nightly (daily), Preview (Tuesdays), Stable (Tuesdays)
Package managers	npm, Homebrew, MacPorts, Anaconda

Installation

```
# Instant run (no install)
npx @google/gemini-cli

# Global install via npm
npm install -g @google/gemini-cli

# Homebrew (macOS/Linux)
brew install gemini-cli

# MacPorts (macOS)
sudo port install gemini-cli

# Release channels
```

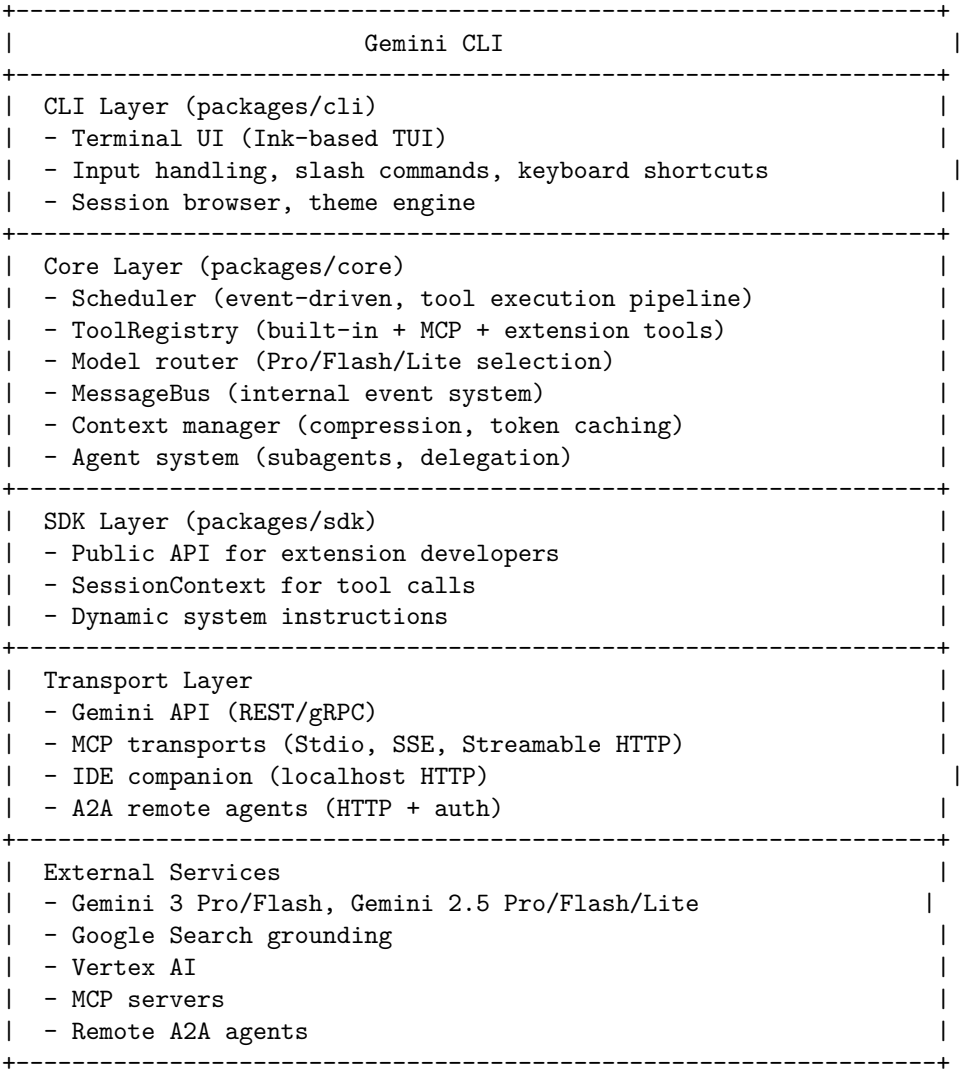
```
npm install -g @google/gemini-cli@latest      # stable
npm install -g @google/gemini-cli@preview     # weekly preview
npm install -g @google/gemini-cli@nightly     # daily nightly
```

Authentication Options

Method	Best For	Setup
Google OAuth	Individual devs, Gemini Code Assist license holders	gemini then choose “Sign in with Google”
Gemini API Key	Specific model control, paid tier	export GEMINI_API_KEY="..."
Vertex AI	Enterprise teams, production workloads	export GOOGLE_API_KEY="..." && export GOOGLE_GENAI_USE_VERTEXAI=true

2. Architecture

High-Level Architecture



Monorepo Package Structure

```
packages/  
cli/           # Terminal UI, slash commands, interactive shell  
core/          # Scheduler, tools, model routing, agents, MCP client  
sdk/           # Public SDK for extensions  
a2a-server/    # Agent-to-Agent protocol server  
devtools/      # DevTools inspector  
test-utils/    # Testing utilities  
vscode-ide-companion/ # VS Code companion extension
```

Key Architectural Decisions

1. **Event-driven scheduler:** Since v0.27.0, tool execution uses an event-driven scheduler for responsive, non-blocking operation. The `MessageBus` is the central nervous system for internal communication.
2. **Model routing:** Gemini CLI intelligently routes between Pro (complex tasks) and Flash (simple queries) to optimize quota usage. This includes automatic switching between Pro for planning and Flash for implementation when Plan Mode model routing is enabled.
3. **Progressive disclosure:** Skills and agent metadata are loaded at startup, but full instructions are injected only on activation to conserve context tokens.
4. **Shadow Git for checkpoints:** File state is captured via commits in a separate Git repository at `~/.gemini/history/<project_hash>`, completely isolated from the user’s own Git repo.

3. Core Features

Code Understanding & Generation

- Query and navigate large codebases using natural language
- Generate new applications from PDFs, images, or sketches (multimodal)
- Debug issues with context from files, shell output, and web searches
- Refactor code with structured planning (Plan Mode)

File Operations

Tool	Kind	Description
read_file	Read	Read single file (text, images, audio, PDF) with line ranges
read_many_files	Read	Read multiple files, triggered by @ syntax
write_file	Edit	Create or overwrite files (requires confirmation)
replace	Edit	Precise text replacement with old/new string matching
glob	Search	Find files by glob pattern
grep_search	Search	Regex search across file contents
list_directory	Read	List directory contents

Shell Execution

```
# Direct shell command (from within Gemini CLI)  
!git status  
  
# Toggle shell mode  
!
```

```
# Background processes
# The run_shell_command tool supports is_background parameter
```

The `run_shell_command` tool supports: - Arbitrary shell commands with manual confirmation - Interactive sessions (vim, rebase -i, etc.) since v0.9.0 - Background processes via `is_background` parameter - Working directory specification via `dir_path` - Sandbox isolation (Docker, Podman, macOS Seatbelt, gVisor, LXC)

Web & Search

Tool	Description
<code>google_web_search</code>	Google Search grounding for real-time information
<code>web_fetch</code>	Retrieve and process content from URLs (HTML, JSON, raw)

Planning

- **Plan Mode** (enabled by default since v0.34.0): Switches to read-only mode for safe research
- `enter_plan_mode` / `exit_plan_mode` tools for structured planning
- Plans stored as Markdown in `~/.gemini/tmp/<project>/plans/`
- Model routing: automatically uses Pro for planning, Flash for implementation
- Plans can be opened in external editor, reviewed, annotated, and approved before execution
- `/plan copy` to copy approved plan to clipboard

Interaction Tools

Tool	Description
<code>ask_user</code>	Request clarification via interactive dialog
<code>write_todos</code>	Internal task tracking displayed to user
<code>save_memory</code>	Persist facts to GEMINI.md
<code>activate_skill</code>	Load specialized expertise on demand
<code>get_internal_docs</code>	Access Gemini CLI's own documentation
<code>complete_task</code>	Subagent task completion (internal)

Input Shortcuts

Shortcut	Action
<code>@path/to/file</code>	Include file content in prompt
<code>@path/to/dir/</code>	Include directory contents
<code>!command</code>	Execute shell command directly
<code>! (alone)</code>	Toggle shell mode
<code>Ctrl+L</code>	Clear screen
<code>Esc Esc</code>	Rewind through history
<code>Alt+M / Ctrl+M</code>	Toggle raw/rendered markdown
<code>Alt+Z / Cmd+Z</code>	Undo in input
<code>Tab</code>	Switch focus between shell and input

4. Models

Supported Models

Gemini CLI supports multiple Gemini model families with configurable aliases:

Model	Context Window	Notes
Gemini 3 Pro Preview	1M tokens	Default for complex tasks, highest capability
Gemini 3 Flash Preview	1M tokens	Fast, cost-effective, surprisingly capable
Gemini 3.1 Pro Preview	1M tokens	Latest preview (since v0.31.0)
Gemini 2.5 Pro	1M tokens	Previous generation Pro
Gemini 2.5 Flash	1M tokens	Previous generation Flash
Gemini 2.5 Flash Lite	Smaller	Used internally for classifiers, summarizers

Model Routing & Aliases

Gemini CLI uses a sophisticated model aliasing and routing system:

```
{
  "modelConfigs": {
    "aliases": {
      "chat-base-3": {
        "extends": "chat-base",
        "modelConfig": {
          "generateContentConfig": {
            "thinkingConfig": { "thinkingLevel": "HIGH" }
          }
        }
      },
      "classifier": {
        "extends": "base",
        "modelConfig": {
          "model": "gemini-2.5-flash-lite",
          "generateContentConfig": {
            "maxOutputTokens": 1024,
            "thinkingConfig": { "thinkingBudget": 512 }
          }
        }
      },
      "summarizer-default": {
        "extends": "base",
        "modelConfig": {
          "model": "gemini-2.5-flash-lite",
          "generateContentConfig": { "maxOutputTokens": 2000 }
        }
      }
    }
  }
}
```

Internal model roles: - **chat-base-3** / **chat-base-2.5**: Main conversation models - **classifier**: Task classification and routing (Flash Lite, low budget) - **prompt-completion**: Autocomplete suggestions (Flash Lite, no thinking) - **fast-ack-helper**: Quick acknowledgments (Flash Lite, 120 max tokens) - **edit-corrector**: Edit verification (Flash Lite, no thinking) - **summarizer-default** / **summarizer-shell**: Tool output summarization - **web-search**: Google Search grounding (Flash base + googleSearch tool)

Thinking Configuration

Models support configurable thinking: - **Gemini 3 models**: **thinkingLevel**: "HIGH", "MEDIUM", "LOW", "OFF" - **Gemini 2.5 models**: **thinkingBudget**: numeric token count (e.g., 8192)

Selecting a Model

Command line

```
gemini -m gemini-2.5-flash
```

Interactive

```
/model set gemini-3-pro-preview
```

Persistent (in settings.json)

```
{ "model": { "name": "gemini-3-flash-preview" } }
```

Pricing & Quotas

Auth Method	Free Tier
Google OAuth	60 req/min, 1,000 req/day
Gemini API Key	1,000 req/day (mix of Flash and Pro)
Vertex AI	Usage-based billing, higher limits

Use `/stats model` to check quota usage. The `/upgrade` command opens the upgrade page for higher limits.

5. Tool System

Architecture

The `ToolRegistry` class manages all available tools. Tools are categorized by kind:

Tool Kinds:

- Execute - Shell commands (mutating, requires confirmation)
- Edit - File writes/replacements (mutating, requires confirmation)
- Read - File reads, directory listing (safe, auto-approved)
- Search - Glob, grep, web search (safe, auto-approved)
- Fetch - Web content retrieval
- Plan - Plan mode entry/exit
- Think - Memory, internal docs access
- Communicate - User interaction (`ask_user`)
- Other - Todos, skill activation, task completion

Complete Built-in Tools Reference

Tool	Kind	Parameters
<code>run_shell_command</code>	Execute	<code>command</code> , <code>description</code> , <code>dir_path</code> , <code>is_background</code>
<code>glob</code>	Search	<code>pattern</code> , <code>dir_path</code> , <code>case_sensitive</code> , <code>respect_git_ignore</code> , <code>respect_gemini_ignore</code>
<code>grep_search</code>	Search	<code>pattern</code> , <code>dir_path</code> , <code>include</code> , <code>exclude_pattern</code> , <code>names_only</code> , <code>max_matches_per_file</code> , <code>total_max_matches</code>
<code>list_directory</code>	Read	<code>dir_path</code> , <code>ignore</code> , <code>file_filtering_options</code>
<code>read_file</code>	Read	<code>file_path</code> , <code>start_line</code> , <code>end_line</code>

Tool	Kind	Parameters
read_many_files	Read	include, exclude, recursive, useDefaultExcludes, file_filtering_options
replace	Edit	file_path, instruction, old_string, new_string, allow_multiple
write_file	Edit	file_path, content
ask_user	Communicate	questions
write_todos	Other	todos
activate_skill	Other	name
get_internal_docs	Think	path
save_memory	Think	fact
enter_plan_mode	Plan	reason
exit_plan_mode	Plan	plan_path
google_web_search	Search	query
web_fetch	Fetch	prompt
complete_task	Other (internal)	result

Tool Confirmation Behavior

- **Mutating tools** (Execute, Edit): Always require manual confirmation. The CLI shows diffs or exact commands before approval.
- **Read-only tools** (Read, Search, Think): Auto-approved.
- **YOLO mode** (`--yolo` or `--approval-mode=yolo`): Auto-approves everything. Only available via command line flag.
- **Auto-edit mode** (`--approval-mode=auto_edit`): Auto-approves edit tools but still confirms shell commands.

Tool Discovery & Management

```
# List all active tools
/tools

# List with descriptions
/tools desc

# Extend via settings
{
  "tools": {
    "discoveryCommand": "path/to/custom/tool-discoverer"
  }
}
```

Manually-Triggered Tool Shortcuts

- `@path` triggers `read_many_files` for file injection
- `!command` triggers `run_shell_command` for direct execution

6. MCP Support

Gemini CLI has first-class MCP (Model Context Protocol) support, positioning itself as both an MCP client and listed under the `mcp-client` and `mcp-server` GitHub topics.

Integration Architecture

```
+-----+          MCP Protocol          +-----+
```

Gemini CLI	<----->	MCP Server
Discovery Layer (mcp-client.ts)	1. List tools 2. List resources 3. List prompts	Exposes: - Tools - Resources
Execution Layer (mcp-tool.ts)	4. Call tools 5. Read resources	- Prompts

Transport Mechanisms

Transport	Description
Stdio	Spawns subprocess, communicates via stdin/stdout
SSE	Connects to Server-Sent Events endpoints
Streamable HTTP	HTTP streaming for communication

Configuration

MCP servers are configured in `~/.gemini/settings.json` or `.gemini/settings.json`:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "$GITHUB_TOKEN"
      },
      "timeout": 30000,
      "trust": false
    },
    "remote-api": {
      "url": "http://localhost:8080/sse",
      "headers": { "Authorization": "Bearer $API_TOKEN" }
    },
    "streaming-service": {
      "httpUrl": "https://api.example.com/mcp",
      "headers": { "X-API-Key": "$SERVICE_KEY" }
    }
  }
}
```

MCP Server Properties

Property	Type	Description
<code>command</code>	string	Executable path for Stdio transport
<code>url</code>	string	SSE endpoint URL
<code>httpUrl</code>	string	HTTP streaming endpoint URL
<code>args</code>	string[]	Command-line arguments (Stdio)
<code>env</code>	object	Environment variables (supports \$VAR / \${VAR} / %VAR%)
<code>cwd</code>	string	Working directory (Stdio)
<code>timeout</code>	number	Request timeout in ms (default: 600,000 = 10 min)

Property	Type	Description
trust	boolean	Bypass all confirmation prompts for this server
includeTools	string[]	Allowlist of tool names to include
excludeTools	string[]	Blocklist of tool names to exclude (takes precedence)
targetAudience	string	OAuth Client ID for IAP-protected apps
targetServiceAccount	string	Service Account email for impersonation

Global MCP Settings

```
{
  "mcp": {
    "allowed": ["my-trusted-server"],
    "excluded": ["experimental-server"],
    "serverCommand": "path/to/global/mcp-server"
  }
}
```

MCP Resources

MCP servers can expose contextual resources in addition to tools. Reference them with the @ syntax:

@server://resource/path

Resources appear in the completion menu alongside filesystem paths.

MCP Management Commands

```
/mcp list          # List servers and tools
/mcp desc          # List with descriptions
/mcp schema        # List with full schemas
/mcp auth <name>   # OAuth authentication flow
/mcp disable <name>
/mcp enable <name>
/mcp reload        # Rediscover all servers
```

MCP Invocation in Prompts

```
> @github List my open pull requests
> @slack Send a summary of today's commits to #dev channel
> @database Run a query to find inactive users
```

7. Skills & Extensions

Agent Skills

Skills are Gemini CLI's equivalent of Claude Code's custom slash commands and skills – on-demand expertise packages that the model can activate when needed.

How Skills Work

1. **Discovery:** At session start, skill names and descriptions are injected into the system prompt
2. **Activation:** When the model identifies a matching task, it calls `activate_skill`
3. **Consent:** User sees a confirmation prompt with skill name, purpose, and directory access

4. **Injection:** SKILL.md body and folder structure are added to conversation context
5. **Execution:** Model proceeds with specialized expertise active

Skill Discovery Tiers

Tier	Location	Scope
Workspace	.gemini/skills/ or .agents/skills/	Project-specific, version controlled
User	~/.gemini/skills/ or ~/.agents/skills/	Personal, all workspaces
Extension	Inside installed extensions	Bundled with extension packages

Precedence: **Workspace** > **User** > **Extension**. Within the same tier, `.agents/skills/` takes precedence over `.gemini/skills/`.

Creating a Skill A skill is a directory containing a SKILL.md file:

```
.gemini/skills/
  security-audit/
    SKILL.md          # Instructions and metadata
    checklist.md      # Bundled resource
    scan-template.sh  # Bundled script
```

The SKILL.md header contains metadata:

```
---
name: security-audit
description: Performs a comprehensive security audit of the codebase
---

# Security Audit Skill

## Instructions
1. Scan for common vulnerabilities...
2. Check dependency versions...
```

Skill Management

```
# Interactive
/skills list
/skills enable <name>
/skills disable <name>
/skills reload

# Terminal
gemini skills list
gemini skills install https://github.com/user/repo.git
gemini skills install /path/to/local/skill
gemini skills install /path/to/my-expertise.skill
gemini skills install https://github.com/org/repo.git --path skills/frontend
gemini skills link /path/to/skills-repo
gemini skills link /path/to/skills-repo --scope workspace
gemini skills uninstall my-expertise --scope workspace
gemini skills enable my-expertise
gemini skills disable my-expertise --scope workspace
```

Built-in Skills

- **pr-creator**: Generates pull request descriptions
- **skill-creator**: Helps create new skills

Extensions

Extensions are a higher-level packaging format that can bundle MCP servers, custom commands, themes, hooks, subagents, and agent skills into a single installable unit.

Extension Management

Install from GitHub

```
gemini extensions install https://github.com/gemini-cli-extensions/workspace
```

Install from local path

```
gemini extensions install /path/to/extension
```

Management

```
gemini extensions list
gemini extensions update <name>|--all
gemini extensions enable <name>
gemini extensions disable <name>
gemini extensions uninstall <name>
gemini extensions restart
```

Interactive commands:

```
/extensions list
/extensions install <url>
/extensions enable <name>
/extensions disable <name>
/extensions explore    # Opens extension gallery in browser
/extensions config <name>
```

Notable Extensions

Extension	Description
Jules	Orchestrate Google’s Jules remote coding agent
Conductor	Advanced planning workflow (“measure twice, implement once”)
Google Workspace	Docs, Slides, Sheets, Chat integration
Hugging Face	Access HF hub models and datasets
Monday.com	Sprint analysis, task board management
Eleven Labs	Audio creation and management
Browserbase	Web page interaction and automation
Endor Labs	Vulnerability scanning, dependency checks
Redis	Natural language Redis data management
Rill	Data analysis with natural language
Data Commons	Public statistical data queries
Arize	AI application instrumentation
Chronosphere	Logs, metrics, traces retrieval

Custom Commands

Custom commands are TOML-based prompt templates stored in the commands directory:

```
~/gemini/commands/    # Global commands
.gemini/commands/     # Project-specific commands
```

TOML Command Format

```
# ~/.gemini/commands/git/commit.toml
# Invoked via: /git:commit

description = "Generates a Git commit message based on staged changes."

prompt = ""
Please generate a Conventional Commit message based on the following git diff:

```diff
!{git diff --staged}
""
```

#### Command Features

Feature	Syntax	Description
Argument injection	{args}	Replace with user-provided text
Shell injection	!{command}	Execute shell, inject output
File injection	@{path}	Embed file content
Namespacing	Directory structure	/git:commit from git/commit.toml
Shell escaping	{args} inside !{}	Auto-escaped for safety

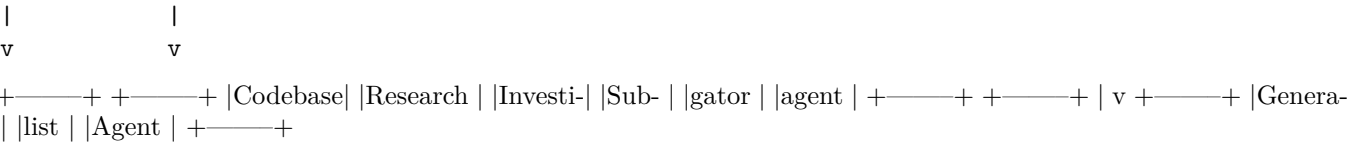
---

## 8. Agent Capabilities

### Subagent Architecture

Gemini CLI has an experimental multi-agent system (requires experimental.enableAgents: true):

Main Agent (Gemini 3 Pro)
---------------------------



### Built-in Agents

- **Codebase Investigator** (since v0.12.0): Explores workspace and resolves relevant information to improve
- **Generalist Agent** (since v0.26.0/v0.32.0): Improves task delegation and routing.
- **Research Subagent** (Plan Mode): Researches context during planning phase.
- **Browser Agent** (experimental, since v0.31.0): Interacts with web pages.
- **CLI Help Agent**: Provides help about Gemini CLI itself.

### Agent Management

/agents list # List all agents (built-in, local, remote) /agents reload # Rescan agent directories /agents enable /agents disable /agents config # Configure model, temperature, limits

### Remote Agents (A2A Protocol)

Since v0.33.0, Gemini CLI supports the Agent-to-Agent (A2A) protocol:

- HTTP authentication for remote agents
- Authenticated A2A agent card discovery
- The `packages/a2a-server` package provides the server implementation
- Remote agents can be discovered from `~/.gemini/agents` and `.gemini/agents` directories

### ### Task Delegation

The `complete\_task` tool is used internally by subagents to return results to the parent agent. The main agent

---

## ## 9. IDE Integration

### ### Supported IDEs

IDE	Support Level
**VS Code**	Full support (companion extension)
**Antigravity** (Google)	Full support
**Cursor, Windsurf**	Terminal keybinding setup via `/terminal-setup`
**Positron**	Supported since v0.28.0
**Zed**	Supported since v0.19.0
**VS Code forks**	Via Open VSX Registry

### ### Features

- **Workspace context:** 10 most recently accessed files, active cursor position, selected text (up to 16KB)
- **Native diffing:** Code changes open in IDE's native diff viewer for review/accept/reject
- **VS Code commands:**
  - `Gemini CLI: Run` -- Start new session
  - `Gemini CLI: Accept Diff` -- Accept changes in diff editor
  - `Gemini CLI: Close Diff Editor` -- Reject changes

### ### Setup

```

```bash
# Automatic (recommended) -- CLI auto-detects IDE and prompts
gemini

# Manual from CLI
/ide install

# Manual from marketplace
# VS Code Marketplace: google.gemini-cli-vscode-ide-companion
# Open VSX Registry: google/gemini-cli-vscode-ide-companion

```

Management

```

/ide enable    # Enable integration
/ide disable   # Disable integration
/ide status    # Check connection status
/ide install   # Install companion extension

```

IDE Companion Spec

The companion extension communicates via a localhost HTTP connection. Environment variables:

- GEMINI_CLI_IDE_WORKSPACE_PATH – workspace directory
- GEMINI_CLI_IDE_SERVER_PORT – companion server port
- GEMINI_CLI_IDE_PID – manual PID override for standalone terminals

Works with Docker containers via `host.docker.internal`.

10. Session Management

Automatic Session Saving

All conversations are automatically saved as you interact. What is saved: - Complete conversation history (prompts + responses) - All tool executions (inputs + outputs) - Token usage statistics - Assistant thoughts and reasoning summaries

Storage location: `~/.gemini/tmp/<project_hash>/chats/`

Sessions are project-specific – switching directories switches session history.

Resuming Sessions

```
# From command line
gemini --resume           # Resume latest session
gemini --resume 1         # Resume by index
gemini --resume a1b2c3d4  # Resume by UUID

# Interactive session browser
/resume                   # Opens interactive browser with search, preview, sort
```

Session Browser Features

- Browse past sessions with timestamps and message counts
- Preview first user prompt for context
- Search across sessions with / key
- Sort by date or message count
- Delete unwanted sessions with x key

Manual Checkpoints

```
/resume save decision-point  # Tag a save point
/resume list                  # List tagged checkpoints
/resume resume decision-point # Restore to tag
/resume delete decision-point # Remove tag
/resume share file.md         # Export to Markdown/JSON
/resume debug                  # Export last API request as JSON
```

Aliases: `/chat` provides identical behavior to `/resume`.

Session Retention

```
{
  "general": {
    "sessionRetention": {
      "enabled": true,
      "maxAge": "30d",
      "maxCount": 50,
      "minRetention": "1d"
    }
  }
}
```

Default: 30-day retention, enabled by default.

Session Limits

```
{
  "model": {
    "maxSessionTurns": 100
  }
}
```

When limit reached: interactive mode shows a message, non-interactive exits with error.

Checkpointing (File State Snapshots)

Separate from session management, checkpointing captures file state before mutations:

```
{
  "general": {
    "checkpointing": { "enabled": true }
  }
}
```

- Creates shadow Git commits at ~/.gemini/history/<project_hash>
- Saves conversation history + tool call that triggered the change
- /restore lists and restores checkpoints
- Restoring reverts files AND conversation, re-proposes the original tool call

Parallel Sessions with Git Worktrees

Gemini CLI supports Git worktrees for running multiple sessions on the same repo without file conflicts.

11. Configuration

Configuration Hierarchy (ascending precedence)

1. Default values (hardcoded)
2. System defaults file (/etc/gemini-cli/system-defaults.json or platform equivalent)
3. User settings (~/.gemini/settings.json)
4. Project settings (.gemini/settings.json)
5. System settings (/etc/gemini-cli/settings.json -- admin overrides)
6. Environment variables (.env files)
7. Command-line arguments

Settings File Locations

File	Location	Scope
System defaults	/etc/gemini-cli/system-defaults.json (Linux), /Library/Application Support/GeminiCli/system-defaults.json (macOS)	Base layer, lowest precedence
User settings	~/.gemini/settings.json	All sessions for current user
Project settings	.gemini/settings.json	Current project only
System overrides	/etc/gemini-cli/settings.json	Admin overrides, highest file precedence

Key Settings Categories

General

```

{
  "general": {
    "preferredEditor": "code",
    "vimMode": false,
    "defaultApprovalMode": "default",
    "devtools": false,
    "enableAutoUpdate": true,
    "enableNotifications": false,
    "checkpointing": { "enabled": false },
    "plan": {
      "modelRouting": true,
      "directory": null
    },
    "maxAttempts": 10,
    "retryFetchErrors": true,
    "sessionRetention": {
      "enabled": true,
      "maxAge": "30d"
    }
  }
}

```

Model

```

{
  "model": {
    "name": "gemini-3-pro-preview",
    "maxSessionTurns": -1,
    "compressionThreshold": 0.5,
    "disableLoopDetection": false,
    "summarizeToolOutput": {
      "run_shell_command": { "tokenBudget": 2000 }
    }
  }
}

```

UI

```

{
  "ui": {
    "theme": "dark",
    "autoThemeSwitching": true,
    "inlineThinkingMode": "off",
    "vimMode": false,
    "showLineNumbers": true,
    "showSpinner": true,
    "loadingPhrases": "tips",
    "hideFooter": false,
    "hideBanner": false,
    "accessibility": {
      "screenReader": false
    }
  }
}

```

Tools

```
{
  "tools": {
    "sandbox": "docker",
    "discoveryCommand": "path/to/custom-tool-discoverer"
  }
}
```

Context

```
{
  "context": {
    "fileName": ["GEMINI.md"],
    "fileFiltering": { }
  }
}
```

Security

```
{
  "security": {
    "folderTrust": { "enabled": true }
  }
}
```

Billing

```
{
  "billing": {
    "overageStrategy": "ask"
  }
}
```

Environment Variables

Variable	Description
GEMINI_API_KEY	Gemini API key
GOOGLE_API_KEY	Google/Vertex AI API key
GOOGLE_GENAI_USE_VERTEXAI	Enable Vertex AI
GOOGLE_CLOUD_PROJECT	GCP project ID
GEMINI_SANDBOX	Sandbox mode (<code>true</code> , <code>docker</code> , <code>podman</code> , <code>sandbox-exec</code> , <code>runsc</code> , <code>lxc</code>)
SEATBELT_PROFILE	macOS sandbox profile
SANDBOX_FLAGS	Custom Docker/Podman flags
SANDBOX_SET_UID_GID	Linux UID/GID handling
GEMINI_CLI_SYSTEM_DEFAULTS_PATH	Custom system defaults path
GEMINI_CLI_SYSTEM_SETTINGS_PATH	Custom system settings path
GEMINI_CLI_IDE_PID	Manual IDE PID override
GEMINI_CLI=1	Set in subprocess env for detection

The .gemini Directory

```
.gemini/
  settings.json      # Project settings
  GEMINI.md          # Project context/instructions
  commands/         # Custom TOML commands
  skills/           # Project skills
  agents/           # Project agents
```

```
sandbox-macos-custom.sb # Custom macOS sandbox profile
sandbox.Dockerfile      # Custom Docker sandbox
.env                    # Project environment variables
```

User-level equivalent: `~/gemini/`

12. Permissions & Security

Approval Modes

Mode	Behavior	How to Set
default	Prompt for all mutating tools	Default behavior
auto_edit	Auto-approve file edits, prompt for shell	<code>--approval-mode=auto_edit</code>
plan	Read-only mode, no mutations	<code>/plan</code> or <code>--approval-mode=plan</code>
yolo	Auto-approve everything	<code>--yolo</code> or <code>--approval-mode=yolo</code> (CLI only)

Sandboxing

Gemini CLI offers five sandboxing methods:

1. macOS Seatbelt (macOS only) Built-in, lightweight. Profiles: - `permissive-open` (default): Write restrictions, network allowed - `permissive-proxied`: Write restrictions, proxied network - `restrictive-open/proxied`: Strict restrictions - `strict-open/proxied`: Read AND write restrictions

2. Docker/Podman (cross-platform) Full container isolation:

```
gemini -s -p "analyze code"
# or
export GEMINI_SANDBOX=docker
```

3. gVisor/runsc (Linux only) Strongest isolation – user-space kernel:

```
export GEMINI_SANDBOX=runsc
gemini -p "run tests"
```

4. Windows Native Sandbox (Windows only) Uses `icaccls` for integrity level enforcement.

5. LXC/LXD (Linux, experimental) Full system containers with `systemd`, `snappy`:

```
lxc launch ubuntu:24.04 gemini-sandbox
export GEMINI_SANDBOX=lxc
gemini -p "build the project"
```

Trusted Folders

When enabled (`security.folderTrust.enabled: true`):

1. First-time folder access triggers a trust dialog
2. Discovery phase scans for commands, MCP servers, hooks, skills, setting overrides
3. Security warnings highlight dangerous configurations
4. Untrusted folders run in restricted “safe mode”:
 - No project settings loaded
 - No `.env` files loaded
 - No extension management
 - No auto-acceptance

- No automatic memory loading
- No MCP server connections
- No custom commands

Trust rules stored in `~/.gemini/trustedFolders.json`.

Policy Engine (since v0.18.0)

Fine-grained policy control for tool execution: - User-defined policies via `--policy` flag - Project-level policies - Admin policies via `adminPolicyPaths` - MCP server wildcards - Tool annotation matching - Shell command allowlisting - Mode-specific policies

```
/policies list    # View active policies
```

Hooks (Lifecycle Events)

Hooks intercept and customize CLI behavior at specific events:

```
/hooks list
/hooks enable <name>
/hooks disable <name>
/hooks enable-all
/hooks disable-all
```

13. Context Management

Context File Hierarchy (GEMINI.md)

Gemini CLI loads instructional context from multiple locations in order:

1. **Global:** `~/.gemini/GEMINI.md` – defaults for all projects
2. **Environment/Workspace:** `GEMINI.md` files in workspace directories and parent directories
3. **Just-in-time (JIT):** When tools access files, the CLI scans for `GEMINI.md` in that directory and ancestors up to trusted root

All found files are concatenated and sent with every prompt.

Example GEMINI.md

```
# Project: My TypeScript Library

## General Instructions
- Follow existing coding style for new TypeScript code
- Ensure all new functions have JSDoc comments
- Prefer functional programming paradigms

## Coding Style
- Use 2 spaces for indentation
- Prefix interface names with `I`
- Always use strict equality (`===`)
```

Modular Imports

```
# Main GEMINI.md

@./components/instructions.md
@../shared/style-guide.md
```

Custom Context File Names

```
{
  "context": {
    "fileName": ["AGENTS.md", "CONTEXT.md", "GEMINI.md"]
  }
}
```

Memory Management

```
/memory show      # Display all loaded context
/memory list       # Show paths of loaded GEMINI.md files
/memory refresh    # Reload from all locations
/memory add <text> # Append to global ~/.gemini/GEMINI.md
```

The `save_memory` tool allows the model to persist facts to GEMINI.md programmatically.

Context Compression

When context usage reaches the compression threshold (default: 50% of context window), the CLI can compress the conversation:

```
/compress      # Manually replace context with summary
```

Configurable via:

```
{
  "model": {
    "compressionThreshold": 0.5
  }
}
```

Token Caching

Available for API key and Vertex AI users (not OAuth). Automatically reuses previous system instructions and context to reduce tokens processed.

```
/stats      # View token usage and cached token savings
```

Tool Output Summarization

Large tool outputs (especially shell commands) can be automatically summarized:

```
{
  "model": {
    "summarizeToolOutput": {
      "run_shell_command": { "tokenBudget": 2000 }
    }
  }
}
```

.geminiignore

Similar to `.gitignore`, files matching patterns in `.geminiignore` are excluded from context loading and file operations.

14. Unique Features

These are features Gemini CLI has that differentiate it from Claude Code:

1. Google Search Grounding

Built-in `google_web_search` tool provides real-time information via Google Search, deeply integrated at the model level rather than via external tools.

2. Free Tier with No API Key

60 requests/min, 1,000 requests/day with just a Google account login. No credit card or API key required for significant usage.

3. Plan Mode with Model Routing

Automatic model switching: Pro for planning, Flash for implementation. Plans stored as reviewable Markdown artifacts with annotation support.

4. Extension Ecosystem with Gallery

Full extension packaging format (MCP servers + commands + themes + hooks + skills + agents) with a browsable gallery at gemini-cli.com/extensions.

5. Interactive Shell

Since v0.9.0, supports running interactive programs (vim, git rebase -i, etc.) directly within the CLI. Click-to-focus in embedded shell output.

6. Shell Mode Toggle

The `!` prefix toggles a persistent shell mode with distinct visual coloring and indicator. All typed text is interpreted as shell commands until toggled off.

7. Conversation Rewind (`/rewind`)

Navigate backward through conversation history with the ability to revert chat state, code changes, or both independently. Accessible via `Esc Esc`.

8. Multi-Directory Workspace

```
gemini --include-directories ../lib,../docs
/directory add ../other-project
/directory show
```

9. GitHub Action Integration

Official `google-github-actions/run-gemini-cli` for automated PR reviews, issue triage, `@gemini-cli` mentions, and custom workflows.

10. Vim Mode

Full vim keybindings in the input area with NORMAL/INSERT modes, count support, motion commands, and persistent preference.

11. A2A (Agent-to-Agent) Protocol

Native support for remote agents via the A2A protocol with HTTP authentication, making Gemini CLI a hub for distributed agent orchestration.

12. Custom Themes & Auto-Switching

Solarized and custom themes with automatic light/dark switching based on terminal background color detection.

13. Checkpointing with Shadow Git

Automatic snapshots before file mutations using a shadow Git repository, with full `/restore` capability that reverts both files and conversation state.

14. Token Caching

Automatic context caching for API key users to reduce costs on subsequent requests.

15. Google Colab Integration

Pre-installed in Colab for headless notebook usage or interactive terminal sessions.

16. Multimodal Input

Generate apps from PDFs, images, sketches. Drag-and-drop multiple files. Paste images from clipboard (Windows, Linux/Wayland/X11, macOS).

17. `gemini-wrapped`

Run `npx gemini-wrapped` for a visual summary of your usage stats, top models, and languages.

15. Limitations vs Claude Code

What Gemini CLI Lacks Compared to Claude Code

Feature	Claude Code	Gemini CLI
Parallel tool execution	Native parallel tool calls (multiple tools in one response)	Sequential tool execution
Background agents	Full background agent support with worktree isolation	On roadmap, not yet shipped
Reasoning effort control	<code>--reasoning-effort</code> flag (low/medium/high)	Thinking level config exists but less user-facing
Worktree isolation	Automatic git worktree creation for isolated agent work	Manual git worktree support
Auto-memory persistence	Automatic memory that persists across sessions	Manual <code>save_memory</code> + <code>GEMINI.md</code>
Hooks with code execution	Pre/post hooks with shell command execution	Hooks system exists but less documented
OAuth provider agnostic	Works with multiple LLM providers via API keys	Locked to Google/Gemini models only
Multi-provider model support	Anthropic, OpenAI, Google models	Gemini models only
Compact UI by default	Clean, minimal terminal UI	Richer but more complex TUI
Edit tool precision	Exact string match required (prevents wrong edits)	<code>replace</code> tool uses <code>old_string/new_string</code> but also has <code>instruction</code> param
Remote MCP via URL	Standard MCP URL support	Full MCP support with SSE, HTTP, Stdio

Where Gemini CLI Excels Over Claude Code

Feature	Gemini CLI Advantage
Free tier	1,000 req/day free with Google account (Claude Code requires API credits)
Context window	1M tokens natively (Claude Code: 200K standard)

Feature	Gemini CLI Advantage
Google Search	Built-in web search grounding
Extension ecosystem	Full extension gallery with installable packages
Interactive shell	Run vim, rebase -i, etc. inside the CLI
Plan Mode	Structured planning with model routing
IDE diffing	Native diff viewer in VS Code
Open source	Apache 2.0 (Claude Code is source-available but not open source)
GitHub Actions	Official action for CI/CD integration
Multimodal	Image/PDF/audio/video input + generation via MCP extensions
Vim mode	Full vim keybindings in input

16. Version History

Timeline of Key Releases

Version	Date	Key Features
v0.1.0	2025-06-25	Initial public release at Google I/O
v0.5.0	~2025-08	Session management, basic MCP support
v0.8.0	2025-09-29	Extensions system launched, new documentation site
v0.9.0	2025-10-06	Interactive shell (vim, rebase -i support)
v0.10.0	2025-10-13	Interactive shell tool calling, polish
v0.11.0	2025-10-20	Jules extension, stream JSON output, markdown toggle
v0.12.0	2025-10-27	Codebase investigator subagent, model routing, model selection
v0.15.0	2025-11-03	Scrollable UI, mouse support, todo planning
v0.16.0	2025-11-10	Gemini 3 launch
v0.18.0	2025-11-17	Policy engine, Google Workspace extension
v0.19.0	2025-11-24	Zed integration, click-to-focus shell
v0.20.0	2025-12-01	Multi-file drag & drop, persistent allow policies
v0.21.0	2025-12-15	Gemini 3 Flash
v0.22.0	2025-12-22	Free tier Gemini 3, Colab integration, Conductor extension
v0.23.0	2026-01-07	Experimental Agent Skills, gemini-wrapped
v0.24.0	2026-01-14	Agent Skills docs, remote agents, folder trust, MessageBus Phase 3
v0.25.0	2026-01-20	Skills enabled by default, pr-creator skill, CLI help agent
v0.26.0	2026-01-27	skill-creator skill, generalist agent, /rewind command
v0.27.0	2026-02-03	Event-driven scheduler, /rewind, Linux clipboard
v0.28.0	2026-02-10	Positron IDE, custom themes, auto theme switching

Version	Date	Key Features
v0.29.0	2026-02-17	Plan Mode, Gemini 3 default for all, extension exploration
v0.30.0	2026-02-25	SDK package, custom skills, policy engine enhancements, vim improvements
v0.31.0	2026-02-27	Gemini 3.1 Pro Preview, experimental browser agent, direct web fetch
v0.32.0	2026-03-03	Generalist agent enabled, model steering, plan editor, shell autocompletion
v0.33.0	2026-03-11	A2A HTTP auth, Plan Mode research subagents, 30-day session retention
v0.34.0	2026-03-17	Plan Mode enabled by default, gVisor + LXC sandboxing
v0.35.0	2026-03-18	Current stable
v0.36.0-preview.0	2026-03-24	Latest preview

Key Milestones

2025-04-17 Repository created (private)
 2025-06-25 Public launch at Google I/O
 2025-09-29 Extensions ecosystem launched
 2025-10-27 Multi-agent architecture (codebase investigator)
 2025-11-10 Gemini 3 model support
 2025-11-17 Policy engine for enterprise
 2025-12-22 Free Gemini 3 for all + Colab
 2026-01-07 Agent Skills system
 2026-02-17 Plan Mode
 2026-02-25 SDK for extension developers
 2026-03-11 A2A remote agents
 2026-03-17 Plan Mode default + advanced sandboxing

Release Cadence

Channel	Schedule	Stability
Nightly	Daily at UTC 00:00	Unvetted, may have issues
Preview	Tuesdays at UTC 23:59	Early feedback, not fully vetted
Stable	Tuesdays at UTC 20:00	Full promotion of prior week's preview + fixes

Appendix A: Quick Command Reference

Slash Commands

Command	Description
<code>/about</code>	Version info
<code>/agents</code>	Manage subagents (experimental)
<code>/auth</code>	Change authentication
<code>/bug</code>	File a bug report

Command	Description
/chat / /resume	Session management and checkpoints
/clear	Clear screen (Ctrl+L)
/commands reload	Reload custom commands
/compress	Compress context to summary
/copy	Copy last output to clipboard
/directory	Manage workspace directories
/docs	Open documentation
/editor	Select editor
/extensions	Manage extensions
/help / /?	Display help
/hooks	Manage lifecycle hooks
/ide	Manage IDE integration
/init	Generate GEMINI.md for current directory
/mcp	Manage MCP servers
/memory	Manage GEMINI.md context
/model	Model selection and configuration
/permissions	Folder trust settings
/plan	Enter/view Plan Mode
/policies	View active policies
/privacy	Privacy notice and consent
/quit / /exit	Exit CLI
/restore	Restore file checkpoints
/rewind	Navigate conversation history (Esc Esc)
/settings	Open settings editor
/shells / /bashes	Toggle background shells view
/setup-github	Set up GitHub Actions
/skills	Manage agent skills
/stats	Session/model/tool statistics
/terminal-setup	Configure terminal keybindings
/theme	Change visual theme
/tools	List available tools
/upgrade	Open upgrade page
/vim	Toggle vim mode

CLI Flags

```

gemini                                # Start interactive
gemini -p "prompt"                    # Non-interactive (headless)
gemini -m gemini-2.5-flash            # Specific model
gemini -s                             # Enable sandboxing
gemini --yolo                         # Auto-approve everything
gemini --resume                      # Resume latest session
gemini --resume 1                     # Resume by index
gemini --list-sessions                # List saved sessions
gemini --delete-session 2             # Delete session
gemini --include-directories ../lib,../docs
gemini --output-format json           # JSON output
gemini --output-format stream-json    # Streaming JSONL
gemini --policy path/to/policy        # Custom policy file
gemini --approval-mode=auto_edit      # Auto-approve edits

```

Exit Codes (Headless Mode)

Code	Meaning
0	Success
1	General error / API failure
42	Input error (invalid prompt)
53	Turn limit exceeded

Appendix B: Comparison Matrix – Gemini CLI vs Claude Code

Dimension	Gemini CLI	Claude Code
Maker	Google (google-gemini.org)	Anthropic
License	Apache 2.0 (fully open source)	Source-available (not open source)
Language	TypeScript	TypeScript
Models	Gemini family only	Claude (Anthropic), also OpenAI, Google via Bedrock/Vertex
Free tier	1,000 req/day with Google account	No free tier (requires API credits or Max subscription)
Context window	1M tokens	200K tokens (standard)
MCP support	Full (Stdio, SSE, HTTP)	Full (Stdio, SSE)
Extension system	Full gallery + installable packages	Skills + MCP (no extension packaging)
IDE integration	VS Code companion + diff viewer	VS Code extension (less native diffing)
Sandboxing	5 methods (Seatbelt, Docker, Podman, gVisor, LXC)	Bash tool sandboxing + permission system
Planning	Structured Plan Mode with artifacts	Think tool + extended thinking
Web search	Built-in Google Search grounding	WebSearch tool (via Brave/etc.)
Background agents	On roadmap	Shipped (with worktree isolation)
Parallel tool calls	Sequential	Native parallel execution
Auto-memory	Manual save_memory + GEMINI.md	Automatic memory persistence
Vim mode	Full vim keybindings	Not available
Interactive shell	Full interactive (vim, etc.)	Non-interactive bash only
GitHub Actions	Official action	Third-party integrations
Headless/scripting	Full JSON/JSONL streaming output	-p flag with text output
Release cadence	Nightly + Preview + Stable weekly	Less frequent, version-based
Community	99K stars, 12.6K forks	Smaller open community

This document was compiled from the official Gemini CLI documentation, GitHub repository, release notes, and published blog posts as of March 25, 2026.